

Department CRUD Generation Flow

This document explains the complete flow for adding a new **Department** CRUD module using the existing **CodeGen full-stack generator**.

The same flow can be reused for any new table such as **Designation, Project, Category**, etc.

1. Overall Flow

```
Create SQL table
↓
Verify dbo.usp_GetObjectSchemas works
↓
Run CodeGen fullstack endpoint
↓
Generated backend files
↓
Copy backend files into GeneratedCrud backend solution
↓
Merge DependencyInjection.cs registration
↓
Build and test backend from Swagger
↓
Copy generated React files into React app
↓
Add React routes/menu
↓
Run backend + frontend
```

2. Target Projects

The final application has two parts:

```
GeneratedCrud
├── GeneratedCrud.Api
├── GeneratedCrud.Domain
└── GeneratedCrud.Infrastructure

GeneratedCrud.React
├── src
│   └── features
```

The generator project produces files into:

```
CodeGen.GenerationFiles/GeneratedOutput
```

3. Create Department Table

Run this script in the same database used by your backend API.

Example database:

```
USE GeneratedCrudDb;  
GO
```

Create table:

```
CREATE TABLE dbo.tblDepartment  
(  
    Id INT IDENTITY(1,1) NOT NULL CONSTRAINT PK_tblDepartment PRIMARY KEY,  
  
    DepartmentName NVARCHAR(100) NOT NULL,  
    DepartmentCode NVARCHAR(20) NOT NULL,  
    Description NVARCHAR(500) NULL,  
  
    CreatedDate DATETIME2(7) NOT NULL  
        CONSTRAINT DF_tblDepartment_CreatedDate DEFAULT SYSUTCDATETIME(),  
  
    UpdatedDate DATETIME2(7) NULL,  
  
    IsDeleted BIT NOT NULL  
        CONSTRAINT DF_tblDepartment_IsDeleted DEFAULT 0  
);  
GO  
  
CREATE UNIQUE INDEX UX_tblDepartment_DepartmentCode  
ON dbo.tblDepartment(DepartmentCode)  
WHERE IsDeleted = 0;  
GO
```

4. Insert Dummy Data

```
INSERT INTO dbo.tblDepartment  
(  
    DepartmentName,  
    DepartmentCode,  
    Description  
)
```

```
VALUES
('Human Resources', 'HR', 'Handles recruitment, employee records, and HR
operations'),
('Information Technology', 'IT', 'Manages software, hardware, infrastructure, and
support'),
('Finance', 'FIN', 'Handles accounts, payroll, budgets, and financial reporting'),
('Sales', 'SAL', 'Handles customer acquisition and revenue generation'),
('Administration', 'ADM', 'Handles office administration and facility
operations');
GO
```

Verify:

```
SELECT * FROM dbo.tblDepartment;
```

5. Verify Schema Stored Procedure

The generator reads table columns from the database using:

```
dbo.usp_GetObjectSchemas
```

Before running the generator, test it manually:

```
EXEC dbo.usp_GetObjectSchemas @TableName = 'dbo.tblDepartment';
```

Expected columns:

```
Id
DepartmentName
DepartmentCode
Description
CreatedDate
UpdatedDate
IsDeleted
```

The stored procedure should return **two result sets**.

For this simple CRUD generator, both result sets can contain the same columns.

6. Run Fullstack Generator

Start the CodeGen API project.

Open Swagger:

```
http://localhost:5144/swagger
```

Call this endpoint:

```
POST /api/generator/fullstack/generate
```

Use these parameters:

```
tableName = dbo.tblDepartment  
solutionName = GeneratedCrud  
frontendAppName = GeneratedCrud.React
```

Full URL example:

```
http://localhost:5144/api/generator/fullstack/generate?  
tableName=dbo.tblDepartment&solutionName=GeneratedCrud&frontendAppName=GeneratedCr  
ud.React
```

Expected naming:

```
dbo.tblDepartment -> Department  
Department       -> Departments  
Controller       -> DepartmentsController  
API route        -> /api/Departments  
React feature    -> src/features/departments
```

7. Generated Backend Files

After generation, backend files will be created under:

```
CodeGen.GenerationFiles/GeneratedOutput
```

Expected generated backend files:

```
GeneratedOutput  
├── GeneratedCrud.Api
```

```
├── Controllers
│   └── DepartmentsController.cs
├── GeneratedCrud.Domain
│   ├── Models
│   │   └── Department.cs
│   ├── DTOs
│   │   ├── DepartmentDto.cs
│   │   ├── CreateDepartmentDto.cs
│   │   └── UpdateDepartmentDto.cs
│   └── Interfaces
│       └── IDepartmentRepository.cs
├── GeneratedCrud.Infrastructure
│   ├── Data
│   │   └── SqlConnectionFactory.cs
│   ├── Repositories
│   │   └── DepartmentRepository.cs
│   └── DependencyInjection.cs
```

8. Copy Backend Files

8.1 Copy API Controller

From:

```
GeneratedOutput/GeneratedCrud.Api/Controllers/DepartmentsController.cs
```

To:

```
GeneratedCrud.Api/Controllers/DepartmentsController.cs
```

8.2 Copy Domain Files

From:

```
GeneratedOutput/GeneratedCrud.Domain/Models/Department.cs
GeneratedOutput/GeneratedCrud.Domain/DTOs/DepartmentDto.cs
GeneratedOutput/GeneratedCrud.Domain/DTOs/CreateDepartmentDto.cs
GeneratedOutput/GeneratedCrud.Domain/DTOs/UpdateDepartmentDto.cs
GeneratedOutput/GeneratedCrud.Domain/Interfaces/IDepartmentRepository.cs
```

To:

```
GeneratedCrud.Domain/Models/Department.cs
GeneratedCrud.Domain/DTOs/DepartmentDto.cs
GeneratedCrud.Domain/DTOs/CreateDepartmentDto.cs
GeneratedCrud.Domain/DTOs/UpdateDepartmentDto.cs
GeneratedCrud.Domain/Interfaces/IDepartmentRepository.cs
```

8.3 Copy Infrastructure Repository

From:

```
GeneratedOutput/GeneratedCrud.Infrastructure.Repositories/DepartmentRepository.cs
```

To:

```
GeneratedCrud.Infrastructure.Repositories/DepartmentRepository.cs
```

9. Important: Do Not Blindly Overwrite `DependencyInjection.cs`

When adding second module like `Department`, your existing `DependencyInjection.cs` already has Employee registration.

Do **not** blindly replace the file.

Open existing file:

```
GeneratedCrud.Infrastructure/DependencyInjection.cs
```

Make sure both repository registrations exist:

```
services.AddScoped<IEmployeeRepository, EmployeeRepository>();
services.AddScoped<IDepartmentRepository, DepartmentRepository>();
```

Final file should look like this:

```
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using GeneratedCrud.Domain.Interfaces;
using GeneratedCrud.Infrastructure.Data;
using GeneratedCrud.Infrastructure.Repositories;
```

```
namespace GeneratedCrud.Infrastructure;

public static class DependencyInjection
{
    public static IServiceCollection AddInfrastructure(
        this IServiceCollection services,
        IConfiguration configuration)
    {
        services.AddSingleton<ISqlConnectionFactory>(_ =>
            new SqlConnectionFactory(
                configuration.GetConnectionString("DefaultConnection")
                ?? throw new
InvalidOperationException("ConnectionStrings:DefaultConnection is not
configured.")));

        services.AddScoped<IEmployeeRepository, EmployeeRepository>();
        services.AddScoped<IDepartmentRepository, DepartmentRepository>();

        return services;
    }
}
```

This is required so ASP.NET Core can inject `IDepartmentRepository` into `DepartmentsController`.

10. Build and Test Backend

Build the solution in Visual Studio:

```
Build > Build Solution
```

Run:

```
GeneratedCrud.Api
```

Open Swagger:

```
http://localhost:5144/swagger
```

You should see:

```
Employees
Departments
```

Test:

```
GET /api/Departments
```

Test create:

```
POST /api/Departments
```

Sample body:

```
{
  "departmentName": "Operations",
  "departmentCode": "OPS",
  "description": "Handles operational processes"
}
```

11. Generated React Files

After fullstack generation, React files will be created here:

```
GeneratedOutput/GeneratedCrud.React/src/features/departments
```

Expected generated React structure:

```
departments
├── models
│   ├── Department.ts
│   ├── CreateDepartmentRequest.ts
│   └── UpdateDepartmentRequest.ts
├── api
│   └── departmentsApi.ts
├── services
│   └── departmentsService.ts
├── components
│   └── DepartmentForm.tsx
└── pages
    └── DepartmentsListPage.tsx
```



```
|— CreateDepartmentPage.tsx
|— EditDepartmentPage.tsx
```

12. Copy React Files

Copy this folder:

```
GeneratedOutput/GeneratedCrud.React/src/features/departments
```

To:

```
GeneratedCrud.React/src/features/departments
```

Final React features folder:

```
GeneratedCrud.React/src/features
|— employees
|— departments
```

13. Update React Routes

Open:

```
GeneratedCrud.React/src/App.tsx
```

Add imports:

```
import { DepartmentsListPage } from
"./features/departments/pages/DepartmentsListPage";
import { CreateDepartmentPage } from
"./features/departments/pages/CreateDepartmentPage";
import { EditDepartmentPage } from
"./features/departments/pages/EditDepartmentPage";
```

Add menu link:

```
<Link to="/departments">Departments</Link>
```

Add routes:

```
<Route path="/departments" element={<DepartmentsListPage />} />
<Route path="/departments/create" element={<CreateDepartmentPage />} />
<Route path="/departments/:id/edit" element={<EditDepartmentPage />} />
```

Example full `App.tsx`:

```
import { BrowserRouter, Link, Navigate, Route, Routes } from "react-router-dom";
import { EmployeesListPage } from "../features/employees/pages/EmployeesListPage";
import { CreateEmployeePage } from
  "../features/employees/pages/CreateEmployeePage";
import { EditEmployeePage } from "../features/employees/pages/EditEmployeePage";
import { DepartmentsListPage } from
  "../features/departments/pages/DepartmentsListPage";
import { CreateDepartmentPage } from
  "../features/departments/pages/CreateDepartmentPage";
import { EditDepartmentPage } from
  "../features/departments/pages/EditDepartmentPage";
import "../App.css";

function App() {
  return (
    <BrowserRouter>
      <div className="app-shell">
        <header>
          <h1>Simple Employee CRUD</h1>

          <nav>
            <Link to="/employees">Employees</Link>
            <Link to="/departments">Departments</Link>
          </nav>
        </header>

        <main>
          <Routes>
            <Route path="/" element={<Navigate to="/employees" replace />} />

            <Route path="/employees" element={<EmployeesListPage />} />
            <Route path="/employees/create" element={<CreateEmployeePage />} />
            <Route path="/employees/:id/edit" element={<EditEmployeePage />} />

            <Route path="/departments" element={<DepartmentsListPage />} />
            <Route path="/departments/create" element={<CreateDepartmentPage />} />

            <Route path="/departments/:id/edit" element={<EditDepartmentPage />} />
          </Routes>
        </main>
      </div>
```

```
    </BrowserRouter>
  );
}

export default App;
```

14. Check React Environment File

Open or create:

```
GeneratedCrud.React/.env
```

Set backend API base URL:

```
VITE_API_BASE_URL=http://localhost:5144
```

Do not include `/api/Departments` in this value.

Correct:

```
VITE_API_BASE_URL=http://localhost:5144
```

Wrong:

```
VITE_API_BASE_URL=http://localhost:5144/api/Departments
```

The generated API client already adds:

```
/api/Departments
```

15. Confirm Backend CORS

Your ASP.NET Core backend must allow the React dev server.

In:

```
GeneratedCrud.Api/Program.cs
```

Before `builder.Build()`:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("ReactApp", policy =>
    {
        policy
            .WithOrigins("http://localhost:5173")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});
```

After `var app = builder.Build();`:

```
app.UseCors("ReactApp");
```

Make sure `app.UseCors("ReactApp");` appears before `app.MapControllers();`.

16. Run Both Applications

Backend

Run from Visual Studio:

```
GeneratedCrud.Api
```

Backend URL:

```
http://localhost:5144/swagger
```

Frontend

Run from terminal:

```
cd GeneratedCrud.React
npm run dev
```

Frontend URL:

```
http://localhost:5173
```

Open:

```
http://localhost:5173/departments
```

17. Quick Checklist

Use this checklist every time you add a new table.

```
[ ] SQL table created
[ ] Dummy data inserted
[ ] dbo.usp_GetObjectSchemas tested
[ ] Fullstack generator executed
[ ] API controller copied
[ ] Domain model copied
[ ] DTOs copied
[ ] Repository interface copied
[ ] Repository implementation copied
[ ] DependencyInjection.cs merged, not overwritten
[ ] Backend builds
[ ] Swagger endpoint works
[ ] React feature folder copied
[ ] React imports added in App.tsx
[ ] React routes added in App.tsx
[ ] React menu link added
[ ] .env has correct VITE_API_BASE_URL
[ ] Backend CORS allows http://localhost:5173
[ ] React page tested in browser
```

18. Common Mistakes

Mistake 1: Stored procedure not found

Error:

```
Could not find stored procedure 'dbo.usp_GetObjectSchemas'
```

Fix: create `dbo.usp_GetObjectSchemas` in the same database used by the generator connection string.

Mistake 2: Wrong database

The table exists but generator returns no columns.

Fix: check the generator connection string in:

```
CodeGen.Api/appsettings.json
```

Mistake 3: Overwriting `DependencyInjection.cs`

If you overwrite it, old repository registrations may be lost.

Fix: merge registrations manually.

Mistake 4: React CORS error

Browser shows CORS error.

Fix: add CORS policy in ASP.NET Core API for:

```
http://localhost:5173
```

Mistake 5: Wrong API base URL

Do not set:

```
VITE_API_BASE_URL=http://localhost:5144/api/Departments
```

Use only:

```
VITE_API_BASE_URL=http://localhost:5144
```

19. Reusing This Flow for Another Table

For another table such as:

```
dbo.tblDesignation
```

Run:

```
http://localhost:5144/api/generator/fullstack/generate?
tableName=dbo.tblDesignation&solutionName=GeneratedCrud&frontendAppName=GeneratedC
rud.React
```

Then repeat the same copy/merge steps.

Expected generated names:

```
dbo.tblDesignation -> Designation
Controller          -> DesignationsController
React feature       -> src/features/designations
API route           -> /api/Designations
```